Nom i Cognoms:	Christopher Richard Carrillo Crespo
URL Repositori Github:	https://github.com/ChristopherR17/DAM2_Deberes/tree/main/M06/RA6

ACTIVITAT
Objectius: ● Familiaritzar-se amb el desenvolupament d'APIs REST utilitzant Express.js ● Aprendre a integrar serveis de processament de llenguatge natural i visió artificial ● Practicar la implementació de patrons d'accés a dades i gestió de bases de dades ● Desenvolupar habilitats en documentació d'APIs i logging ● Treballar amb formats JSON i processament de dades estructurades
Criteris d'avaluació: - Cada pregunta indica la puntuació corresponent
Entrega: - Repositori git que contingui el codi que resol els exercicis i, en el directori "doc", aquesta memòria resposta amb nom "memoria.pdf"

Punt de partida
https://github.com/jpala4-ieti/DAM-M0486-Tema3-RA6-PR4.2-Punt-Partida-25-26

Preparació de l'activitat

- Clonar el repositori de punt de partida
- Llegir els fitxers README*.md que trobaràs en els diferents directoris
- Assegurar-te de tenir una instància de MySQL/MariaDB funcionant
- Tenir accés a una instància d'Ollama funcionant (al centre te'n facilitem una)

Entrega

- URL de resopitori

Exercicis

Exercici 1 (2.5 punts)

L'objectiu de l'exercici és familiaritzar-te amb **xat-api**. Respon la les preguntes dins el requadre que trobaràs al final de l'exercici.

Configuració i Estructura Bàsica:

1. Per què és important organitzar el codi en una estructura de directoris com controllers/, routes/, models/, etc.? Quins avantatges ofereix aquesta organització?
2. Analitzant el fitxer server.js, quina és la seqüència correcta per inicialitzar una aplicació Express? Per què és important l'ordre dels middlewares?
3. Com gestiona el projecte les variables d'entorn? Quins avantatges ofereix usar dotenv respecte a hardcodejar els valors?

API REST i Express:

1. Observant chatRoutes.js, com s'implementa el routing en Express? Quina és la diferència entre els mètodes HTTP GET i POST i quan s'hauria d'usar cadascun?
2. En el fitxer chatController.js, per què és important separar la lògica del controlador de les rutes? Quins principis de disseny s'apliquen?
3. Com gestiona el projecte els errors HTTP? Analitza el middleware errorHandler.js i explica com centralitza la gestió d'errors.

Documentació amb Swagger:

1. Observant la configuració de Swagger a swagger.js i els comentaris a chatRoutes.js, com s'integra la documentació amb el codi? Quins beneficis aporta aquesta aproximació?
2. Com es documenten els diferents endpoints amb els decoradors de Swagger? Per què és important documentar els paràmetres d'entrada i sortida?
3. Com podem provar els endpoints directament des de la interfície de Swagger? Quins avantatges ofereix això durant el desenvolupament?

Base de Dades i Models:

1. Analitzant els models Conversation.js i Prompt.js, com s'implementen les relacions entre models utilitzant Sequelize? Per què s'utilitza UUID com a clau primària?
2. Com gestiona el projecte les migracions i sincronització de la base de dades? Quins riscos té usar `sync()` en producció?
3. Quins avantatges ofereix usar un ORM com Sequelize respecte a fer consultes SQL directes?

Logging i Monitorització:

1. Observant logger.js, com s'implementa el logging estructurat? Quins nivells de logging existeixen i quan s'hauria d'usar cadascun?
2. Per què és important tenir diferents transports de logging (consola, fitxer)? Com es configuren en el projecte?
3. Com ajuda el logging a debugar problemes en producció? Quina informació crítica s'hauria de loguejar?

Configuració i Estructura Bàsica:

1.

Es importante organizar el código de esa manera porque nos permite separar el código según la responsabilidad de este.

- routes/: define los endpoints y las URLs de la API
- controllers/: contiene la lógica de negocio que se ejecuta cuando se llama a una ruta
- models/: define la estructura de los datos y la interacción con la base de datos mediante Sequelize

Al final, este tipo de estructura nos mejora la legibilidad, el mantenimiento y nos facilita reutilizar código, ya que tenemos cada parte del proyecto bien definida.

2.

La secuencia correcta es la siguiente:

1. Cargar las variables de entorno mediante dotenv.config(), para disponer de la configuración desde el inicio de la aplicación.
2. Importamos las dependencias principales como Express, CORS, Swagger, Sequelize, el logger y las rutas.
3. Crear una instancia de Express(con const app = express())
4. Configuramos los middlewares globales como cors() y express.json(), que nos permiten peticiones de otros dominios y poder hacer peticiones en formato JSON, respectivamente
5. Configuracion del Swagger, exponiendo la documentación en el endpoint
6. Middleware de logging personalizado, para registrar información detallada de cada petición HTTP
7. Middleware de logging específico de Express, que añade información sobre las peticiones y respuestas
8. Registro de las rutas principales(en este caso /api/chat)
9. Middleware de gestión centralizado de errores, que se ejecuta al final de la cadena
10. Inicialización del servidor, donde se:
    - Verifica la conexión con la base de datos
    - Sincronizan los modelos con Sequelize

- Se arranca el servidor HTTP en el puerto configurado

El orden de los middlewares es fundamental porque Express los ejecuta de forma secuencial.

3.

El proyecto gestiona las variables de entorno mediante el paquete dotenv, que carga las variables definidas en el archivo .env a process.env.

El dotenv nos evita tener que hardcodear valores sensibles, facilita la configuración según el entorno de ejecución y mejora la seguridad y el mantenimiento del proyecto.

API REST i Express:

1.

El routing se implementa con express.Router(), asociando cada endpoint a un controlador. GET se utiliza para obtener datos (conversaciones o modelos) y POST para enviar información al servidor (prompts del chat).

2.

Separar rutas y controladores permite que cada parte tenga una responsabilidad concreta. Las rutas definen los endpoints y los controladores gestionan la lógica, aplicando principios como Single Responsibility y la arquitectura MVC.

3.

El proyecto utiliza un middleware de errores centralizado que registra todos los errores y devuelve respuestas HTTP coherentes. Se gestionan errores específicos de Sequelize y errores generales, adaptando la información mostrada según el entorno.

Documentació amb Swagger:

1.

Swagger se integra con swagger-jsdoc que genera la documentación a partir de los comentarios de las rutas. Esto permite tener una documentación automática, actualizada y alineada con el código.

2.

Los endpoints se documentan con anotaciones Swagger en las rutas. Documentar entradas y salidas es clave para entender cómo utilizar la API y evitar errores.

3.

Swagger se integra con swagger-jsdoc que genera la documentación a partir de los comentarios de las rutas. Esto permite tener una documentación automática, actualizada y alineada con el código.

Base de Dades i Models:

1.

La relación se establece utilizando la Foreign Key ConversationId en la tabla prompts, estableciendo una relación 1:N.

Se utiliza el UUID como clave primaria para sostener una buena seguridad entre los datos y su accesibilidad.

2.

La base de datos se gestiona con sequelize, el cual ofrece el método nombrado en la pregunta sync(). Este crea tablas que no existen y dependiendo la forma en que se configuren se pueden reiniciar tablas existentes. Los riesgos identificados en su utilización son la pérdida de datos, cambios no controlados, falta de historial de cambios, etc. En sí, se recomienda usar migraciones controladas en prod

3.

Sequelize permite trabajar con la Base de Datos usando objetos y métodos JS. Gestiona modelos y relaciones de manera intuitiva y también soporta múltiples motores de bases de datos. Añade validaciones y mantiene un flujo de migraciones y sincronización controlado. Todo esto reduce errores y hace el desarrollo más rápido y seguro en comparación con consultas SQL directas

Logging i Monitorització:

1.

La estructura que veremos por logging es la siguiente:

- Timestamp
- Nivel del log (error, info,etc)

- Mensaje
- MetadataStack Trace (en caso de errores)

Los logs se guardan en consolas y archivos rotativos y también existe un middleware que registra las peticiones HTTP.

Los niveles que se encuentran son error, warn, info y debug los cuales permiten clasificar la severidad de los eventos.

2.

Es importante porque permite separar logs por uso, esto garantiza la persistencia, facilita monitorización y dentro del proyecto están configuradas usando transports.Console - transports.DailyRotateFile cada uno con su formato y opciones

3.

Los loggins registran eventos y errores críticos mientras la app sigue funcionando, permitiendo recrear lo que paso antes de un fallo y facilitando encontrar la causa principal de los errores, sin interrumpir el servicio. Esto siempre ayudará a la depuración efectiva
Un ejemplo de uso se puede ver tras una petición HTTP, si un endpoint falla, se podrá ver exactamente qué petición causó el error, entre otras características

Exercici 2 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici2.js**

Modifica el codi per tal que, pels dos primers jocs i les 2 primeres reviews de cada joc, crei una estadística que indiqui el nombre de reviews positives, negatives o neutres.

Modifica el prompt si cal.

Guarda la sortida en el directori data amb el nom **exercici2_resposta.json**

Exemple de sortida

```
{
  "timestamp": "2025-01-09T12:30:45.678Z",
  "games": [
    {
      "appid": "730",
      "name": "Counter-Strike 2",
      "statistics": {
        "positive": 1,
        "negative": 0,
        "neutral": 1,
        "error": 0
      }
    },
    {
      "appid": "570",
      "name": "Dota 2",
      "statistics": {
        "positive": 1,
        "negative": 1,
        "neutral": 0,
        "error": 0
      }
    }
  ]
}
```

Exercici 3 (2.5 punts)

Dins de **practica-codi** trobaràs **src/exercici3.js**

Modifica el codi per tal que retorni un anàlisi detallat sobre l'animal.
Modifica el prompt si cal.

La informació que volem obtenir és:

- Nom de l'animal.
- Classificació taxonòmica (mamífer, au, rèptil, etc.)
- Hàbitat natural
- Dieta
- Característiques físiques (mida, color, trets distintius)
- Estat de conservació

Guarda la sortida en el directori **data** amb el nom **exercici3_resposta.json**

```
{
    "analisis": [
        {
            "imatge": {
                "nom_fitxer": "nom_del_fitxer.jpg",
            },
            "analisi": {
                "nom_comu": "nom comú de l'animal",
                "nom_cientific": "nom científic si és conegut",
                "taxonomia": {
                    "classe": "mamífer/au/rèptil/amfibi/peix",
                    "ordre": "ordre taxonòmic",
                    "familia": "família taxonòmica"
                },
                "habitat": {
                    "tipus": ["tipus d'hàbitats"],
                    "regioGeografica": ["regions on viu"],
                    "clima": ["tipus de climes"]
                },
                "dieta": {
                    "tipus": "carnívor/herbívor/omnívor",
                    "aliments_principals": ["llista d'aliments"]
                },
                "caracteristiques_fisiques": {
                    "mida": {
                        "altura_mitjana_cm": "altura mitjana",
                        "pes_mitja_kg": "pes mitjà"
                    },
                    "colors_predominants": ["colors"],
                    "trets_distintius": ["característiques"]
                },
                "estat_conservacio": {
                    "classificacio_IUCN": "estat",
                    "amenaces_principals": ["amenaces"]
                }
            }
        }
    ]
}
```

Exercici 4 (2.5 punts)

Implementa un nou endpoint a xat-api per realitzar anàlisi de sentiment

Haurà de complir els següents requisits

- Estar disponible a l'endpoint POST /api/chat/sentiment-analysis
- Disposar de documentació swagger
- Emmagatzemar informació a la base de dades
- Usar el logger a fitxer

Abans d'implementar la tasca, explica en el requadre com la plantejaràs i fes una proposta de json d'entrada, de sortida i de com emmagatzemaràs la informació a la base de dades.

- **ENDPOINT:** POST /api/chat/sentiment-analysis

- **JSON de Entrada**
{
"conversationsId": 321,
"text": "Este es el texto a analizar..."
}

- **JSON de Salida**
{
"conversationsId": 321,
"text": "Este es el texto a analizar...",
"sentiment": "negativo",
"timstamp": "2026-02-01T16:30:30Z"
}

- **DB** ID (uuid) conversationsId (UUID) text sentiment createdAt
- **Logger** logger.info('Sentiment analysis', { conversationId, text. sentiment, timestamp: new Date().tolSOString() });
- **Swagger** - Mostrar método utilizado (POST) - Request body (id y text) - Respuestas posibles (201 creado - 400 error)